Autonomous AI-Agent Marketplace for Micro-SaaS Solopreneurs

Project Workbook

By
Nikhil Hiro Ghind
Vineet Kumar
Sakshat Nandkumar Patil

October 28, 2026

Advisor: Professor Rakesh Ranjan

Project summary. The Autonomous AI-Agent Marketplace is a platform on which developers publish purpose-built, autonomous "agentic microservices" (for example: customer-support resolution, dynamic pricing, invoicing and cross-border tax compliance) and on which solopreneurs running micro-SaaS businesses discover, configure, approve and monitor those agents against their own business systems. The project repository is github.com/vineetkumar-sjsu/CMPE_295A_Final_Project_FA26.

Team responsibilities. Each member owns one technical area for the whole project and authored the corresponding workbook sections below.

Member	Primary technical ownership	Workbook sections owned
Nikhil Hiro Ghind	Agent runtime, developer SDK/CLI, reference agents, guardrails & human-in-the-loop, agent evaluation	1.1-1.4 (agents, tools, evaluation), 3.4 (agent requirements), 8.3-8.4; Ch. 6 agent and SDK design; Ch. 7 evaluation methodology
Vineet Kumar	Marketplace back-end services (registry, catalog, orchestration), data layer, cloud infrastructure, CI/CD	1.6-1.7 (platforms, systems), Ch. 5 Architecture, 8.1, Ch. 9 Schedule; Ch. 6 component, data and API design; Ch. 7 deployment plan; Ch. 9 PERT
Sakshat Nandkumar Patil	Web front-ends (Developer Portal, Solopreneur Console), identity & security, billing, observability dashboards	1.5 (security), State of the Art, Ch. 2 Justification, Ch. 3-4, 8.2; Ch. 6 security and UI design; Ch. 7 QA strategy

Workbook 2 update. This submission completes Chapters 6 (Project Design) and 7 (QA, Performance, Deployment Plan), updates Chapters 1, 3, 5, 8 and 9 (including the two agent delivery modes requested by the advisor, FR-26), and reports implementation progress as of October 28, 2026. Changes to Chapters 8 and 9 relative to Workbook 1 are shown in bold.

Chapter 1. Literature Search, State of the Art

Literature Search

The project sits at the intersection of four research areas: (i) large-language-model (LLM) based autonomous agents and their ability to use tools and APIs; (ii) the evaluation and security of such agents when they act on real systems; (iii) software marketplaces and two-sided platforms; and (iv) the cloud-systems techniques (microservices, lightweight isolation) needed to run untrusted third-party code. The review below is organised along those lines and closes with the research gap this project addresses. Bracketed numbers refer to the References section at the end of this chapter.

LLM-based autonomous agents

Two peer-reviewed surveys frame the field. Wang et al. [1] (Frontiers of Computer Science, 2024) propose a unified agent architecture of **profile, memory, planning and action** modules and catalogue applications in social science, natural science and engineering. Xi et al. [2] (Science China Information Sciences, 2025) describe agents as a "brain" (the LLM), "perception" and "action" components and discuss single-agent, multi-agent and human-agent settings. Both identify reliability, long-horizon planning and safe tool use as open problems - exactly the properties a solopreneur needs before delegating business operations. The ReAct paradigm [3] (ICLR 2023) interleaves reasoning traces with actions and is the control loop adopted by most production agent frameworks, including the LangGraph-based runtime planned for this project. Reflexion [10] (NeurIPS 2023) adds verbal self-reflection over failed attempts, improving task success without weight updates; we plan to use a similar critique step before an agent commits an irreversible business action.

Tool use and API integration

A marketplace agent is only useful if it can operate the merchant's existing SaaS stack. Toolformer [4] (NeurIPS 2023) showed that LLMs can learn when and how to call external APIs in a self-supervised way. ToolLLM [5] (ICLR 2024) scaled this to more than 16,000 real REST APIs and introduced a depth-first decision-tree search for multi-step tool chains, while Gorilla [6] (NeurIPS 2024) showed that retrieval of up-to-date API documentation sharply reduces hallucinated API calls. HuggingGPT [7] (NeurIPS 2023) is the closest academic analogue to our marketplace: a controller LLM selects and composes specialist models from a public hub based on their textual descriptions. Its results motivate our **machine-readable agent manifest** (capabilities, inputs, permission scopes), which lets both humans and other agents discover and compose marketplace agents. In industry these ideas have been standardised as the Model Context Protocol (MCP) for agent-to-tool connections [24] and the Agent2Agent (A2A) protocol for agent-to-agent delegation [25]; Hou et al. [17] survey the MCP ecosystem and its server life-cycle threats.

Multi-agent orchestration

AutoGen [8] (COLM 2024) provides conversable agents that cooperate through message passing and allow human participation at configurable points, and MetaGPT [9] (ICLR 2024) encodes standard operating procedures into role-specialised agents, reducing cascading hallucinations in multi-step work. Both demonstrate that decomposing work into narrow, specialised agents outperforms one general agent - the core premise of a marketplace of single-purpose agentic microservices. Our orchestrator adopts the explicit-workflow style of MetaGPT for deterministic business processes and AutoGen-style human-in-the-loop checkpoints for high-impact actions.

Evaluating agents on realistic tasks

AgentBench [11] (ICLR 2024) evaluated LLMs across eight interactive environments and found a large gap between commercial and open models, with long-term reasoning and instruction following as the main failure causes. τ-bench [12] (ICLR 2025) is directly relevant to our customer-support and order-management agents: it simulates a user, a domain policy and tool APIs (retail and airline) and introduces the pass^k metric, showing that even state-of-the-art function-calling agents succeed on fewer than half of tasks and are highly inconsistent across repeated trials. This motivates our **certification pipeline**: every published agent must pass a repeated-trial, policy-compliance test suite before listing, and its pass^k score is shown to buyers.

Security of agents and LLM app stores

Greshake et al. [13] (ACM AISec 2023) introduced indirect prompt injection, in which instructions hidden in data retrieved by an agent (an email, a web page, a support ticket) hijack its behaviour. AgentDojo [14] (NeurIPS 2024 Datasets & Benchmarks) offers 97 realistic tasks and 629 security test cases for measuring attacks and defences on tool-using agents. Deng et al. [15] (ACM Computing Surveys, 2025) organise agent threats into unpredictable user input, complex internal execution, variable operational environments and untrusted external entities. Most importantly for a marketplace, Hou, Zhao and Wang [16] (IEEE S&P 2025) analysed LLM app stores and found apps with misleading descriptions, privacy-policy violations, abusive content and malicious behaviour, concluding that stores need stronger vetting. The OWASP Top 10 for LLM Applications [31] adds practical controls such as least-privilege tool scopes and human approval for excessive agency. Our design responds with per-agent permission scopes, sandboxed execution, a policy engine with spending limits, an immutable audit log and pre-listing red-team tests.

Software marketplaces, platform economics and agentic markets

Rochet and Tirole [19] (Journal of the European Economic Association, 2003) model two-sided platforms in which pricing must attract both sides; our marketplace is two-sided (agent developers and solopreneurs), so revenue share and free tiers are design

decisions rather than afterthoughts. Martin et al. [20] (IEEE Transactions on Software Engineering, 2017) survey app-store analysis and show how ratings, reviews and release data feed back into software engineering - signals we expose to developers through usage analytics. Bansal et al. [18] (Microsoft Research, 2025) released Magentic Marketplace, a simulation of markets in which LLM agents act for buyers and sellers; they found that models exhibit a strong first-proposal bias (10-30x advantage for speed over quality) and are sensitive to the search mechanism. This informs our discovery design: ranking is based on certified quality metrics, not just recency. For the dynamic-pricing reference agent, den Boer [23] (Surveys in Operations Research and Management Science, 2015) reviews dynamic pricing with demand learning and provides the baseline (explore-exploit pricing with guardrails) against which an LLM-driven pricing agent can be compared.

Systems foundations

Dragoni et al. [21] describe microservices as small, independently deployable services communicating by messages, and note the matching trade-offs in operational complexity; our marketplace treats each agent as such a service. Firecracker [22] (USENIX NSDI 2020) shows that lightweight virtual machines can give strong isolation with start-up times around 125 ms and low memory overhead, which makes per-tenant isolation of untrusted agent code economically feasible. For the prototype we use gVisor-sandboxed containers on Kubernetes and keep Firecracker microVMs as the hardened option for 295B.

Research gap

Existing academic work studies agents, tool use, evaluation and security largely in isolation, and existing commercial marketplaces (see State of the Art) target enterprises or consumers. No published system combines (a) a developer-facing publishing pipeline with automated certification, (b) least-privilege, sandboxed execution against a merchant's own SaaS accounts, (c) human-in-the-loop approval tuned for a single operator, and (d) transparent reliability metrics (pass^k, cost per task) for buyers who cannot evaluate agents themselves. This project builds and evaluates such a system for the micro-SaaS solopreneur segment.

State-of-the-Art Summary

Industry has moved quickly from chat assistants to marketplaces of agents. OpenAI launched the **GPT Store** in January 2024 [28] for consumer-facing custom GPTs; Salesforce launched **AgentExchange** in March 2025 [27] as a marketplace of agents, actions and templates for its Agentforce platform; and AWS added an **AI Agents and Tools** category to AWS Marketplace in July 2025 [26], alongside AgentCore for running agents. On the protocol side, MCP [24] has become the common way to connect agents to tools, A2A [25] standardises agent discovery and delegation through "Agent Cards", and agent payment protocols - Stripe and OpenAI's Agentic Commerce Protocol (ACP), Google's Agent Payments Protocol (AP2) and Coinbase's x402 - are emerging for

agents that transact on a user's behalf [29]. Developer frameworks (LangGraph, AutoGen, CrewAI, OpenAI Agents SDK) make agents easy to build but leave distribution, trust and billing to the developer.

Table 1. Leading industry offerings compared with this project

Offering	Primary customer	What it provides	Gap for solopreneurs
OpenAI GPT Store [28]	Consumers, prosumers	Catalogue of custom GPTs inside ChatGPT	Conversational, not autonomous; limited back-office integration; weak vetting [16]
Salesforce AgentExchange [27]	Salesforce enterprise customers	Partner agents, actions, templates for Agentforce	Requires the Salesforce platform and enterprise budget
AWS Marketplace AI Agents & Tools [26]	Enterprise buyers on AWS	Procurement of agents, MCP servers and tools through AWS billing	Assumes cloud and DevOps expertise; procurement-oriented
MCP server registries [17][24]	Developers	Discovery of tool servers	Tools, not end-to-end agents; no certification, billing or monitoring
Agent frameworks (LangGraph, AutoGen [8], CrewAI)	Developers	Libraries to build agents	No distribution channel, trust layer or buyer-facing UI
This project	Micro-SaaS solopreneurs and indie developers	Certified agents, sandboxed execution against the merchant's SaaS, HITL approvals, usage billing and reliability metrics	-

Leading-edge techniques we will adopt: MCP for tool access, A2A Agent Cards for capability manifests, policy-guarded tool calling with human approval for high-impact actions, repeated-trial evaluation (pass^k) from τ-bench [12], prompt-injection testing in the style of AgentDojo [14], LLM observability (OpenTelemetry traces, Langfuse) and model routing across providers to control cost.

References

1. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., ... Wen, J.-R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), 186345. https://doi.org/10.1007/s11704-024-40231-1
   Peer-reviewed journal survey; source of the profile/memory/planning/action agent architecture used to structure our agent SDK.

2. Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., ... Gui, T. (2025). The rise and potential of large language model based agents: A survey. *Science China Information Sciences, 68*(2), 121101. https://doi.org/10.1007/s11432-024-4222-0
   Peer-reviewed journal survey covering single-agent, multi-agent and human-agent cooperation; frames our human-in-the-loop design.

3. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
   Introduces the reason-act loop that underlies the agent runtime we will implement.

4. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach

themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS) 36*.

Shows LLMs can decide when and how to call external APIs; foundational to agent tool use.

5. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., ... Sun, M. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*.

   Large-scale real-API tool-use dataset and multi-step search; informs our tool-chaining approach.

6. Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2024). Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS) 37*.

   Retrieval-aware API calling that reduces hallucinated calls; motivates retrieval of connector documentation at run time.

7. Shen, Y., Song, K., Tan, X., Li, D., Lu, W., & Zhuang, Y. (2023). HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems (NeurIPS) 36*.

   An LLM controller selects and composes specialist models from a public hub - the closest academic analogue to a marketplace of composable agents.

8. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., ... Wang, C. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling (COLM)*.

   Multi-agent conversation framework with configurable human participation.

9. Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., ... Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*.

   Encodes standard operating procedures into role-based agents; model for deterministic business workflows.

10. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS) 36*.

    Self-reflection loop that improves agent success; used for our pre-commit critique step.

11. Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., ... Tang, J. (2024). AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*.

    Multi-environment agent benchmark; reference methodology for our evaluation harness.

12. Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2025). τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*. arXiv:2406.12045

    Retail/airline customer-service benchmark with policies and the pass^k reliability metric; basis of our certification tests.

13. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications

with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)* (pp. 79-90). ACM.
Defines indirect prompt injection, the main threat to agents that read customer emails and tickets.

14. Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS 37 Datasets and Benchmarks Track*.
Benchmark of attacks and defences on tool-using agents; template for our red-team suite.

15. Deng, Z., Guo, Y., Han, C., Ma, W., Xiong, J., Wen, S., & Xiang, Y. (2025). AI agents under threat: A survey of key security challenges and future pathways. *ACM Computing Surveys, 57*(7), 1-36. https://doi.org/10.1145/3716628
Peer-reviewed journal taxonomy of agent threats; used to build our threat model.

16. Hou, X., Zhao, Y., & Wang, H. (2025). On the (in)security of LLM app stores. In *IEEE Symposium on Security and Privacy (S&P)* (pp. 298-316). IEEE.
Empirical study of misleading, privacy-violating and malicious apps in LLM app stores; motivates mandatory certification.

17. Hou, X., Zhao, Y., Wang, S., & Wang, H. (2025). Model Context Protocol (MCP): Landscape, security threats, and future research directions. *ACM*. https://doi.org/10.1145/3796519 (arXiv:2503.23278)
Survey of the MCP ecosystem and server life-cycle threats; guides how our connectors are vetted.

18. Bansal, G., Hua, W., Huang, Z., Fourney, A., Swearngin, A., Epperson, W., ... Kambhampati, S. (2025). Magentic Marketplace: An open-source environment for studying agentic markets. arXiv:2510.25779. Microsoft Research.
Simulation of agent-mediated markets showing first-proposal bias and sensitivity to search design.

19. Rochet, J.-C., & Tirole, J. (2003). Platform competition in two-sided markets. *Journal of the European Economic Association, 1*(4), 990-1029.
Classic economics of two-sided platforms; informs developer revenue share and buyer pricing.

20. Martin, W., Sarro, F., Jia, Y., Zhang, Y., & Harman, M. (2017). A survey of app store analysis for software engineering. *IEEE Transactions on Software Engineering, 43*(9), 817-847.
Peer-reviewed survey of how marketplace data (ratings, reviews, releases) informs software engineering.

21. Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In *Present and Ulterior Software Engineering* (pp. 195-216). Springer.
Defines microservices and their trade-offs; the architectural style for each agent and platform service.

22. Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., & Popa, D.-M. (2020). Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)* (pp. 419-434).
Lightweight microVM isolation for multi-tenant serverless code; hardened sandbox option for agents.

23. den Boer, A. V. (2015). Dynamic pricing and learning: Historical origins, current research, and new directions. *Surveys in Operations Research and Management Science, 20*(1), 1-18.
    Peer-reviewed review of dynamic pricing with demand learning; baseline for the pricing reference agent.

24. Anthropic. (2024, November 25). Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol
    Announcement of the open protocol for connecting agents to tools and data sources.

25. Google. (2025, April 9). Announcing the Agent2Agent Protocol (A2A). Google for Developers Blog. https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/
    Open protocol for agent discovery (Agent Cards) and task delegation, now hosted by the Linux Foundation.

26. Amazon Web Services. (2025, July 16). Introducing AI agents and tools in AWS Marketplace. https://aws.amazon.com/about-aws/whats-new/2025/07/ai-agents-tools-aws-marketplace
    Enterprise marketplace category for agents, tools and MCP servers.

27. Salesforce. (2025, March 4). Salesforce launches AgentExchange: The trusted marketplace for Agentforce. https://www.salesforce.com/news/press-releases/2025/03/04/agentexchange-announcement/
    Enterprise agent marketplace tied to the Salesforce platform.

28. OpenAI. (2024, January 10). Introducing the GPT Store. https://openai.com/index/introducing-the-gpt-store/
    Consumer marketplace for custom GPTs.

29. Orium. (2025). Agentic payments explained: ACP, AP2, and x402. https://orium.com/blog/agentic-payments-acp-ap2-x402
    Industry overview of emerging agent payment protocols relevant to agents that transact for merchants.

30. Small Business & Entrepreneurship Council. (2026, June 22). Solopreneur America! (citing U.S. Census Bureau nonemployer statistics). https://sbecouncil.org/2026/06/22/solopreneur-america/
    Market-size data for U.S. nonemployer (solo) businesses used in the project justification.

31. OWASP Foundation. (2025). OWASP Top 10 for LLM Applications 2025. https://genai.owasp.org/llm-top-10/
    Industry security checklist (prompt injection, excessive agency, supply-chain risks) applied to our requirements.

32. Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824-836. https://doi.org/10.1109/TPAMI.2018.2889473
    Peer-reviewed journal article defining the HNSW index; the approximate-search treatment compared with exact search in evaluation baseline B1 (Chapter 7).

33. Brooke, J. (1996). SUS: A 'quick and dirty' usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), Usability evaluation in industry (pp. 189-194). Taylor & Francis.
Standard System Usability Scale used for the usability study and NFR-09.

Chapter 2. Project Justification

The problem. Solo founders are now a large part of the economy: the U.S. Census Bureau counts about 30 million nonemployer businesses, which generated roughly $1.7 trillion in receipts in 2022 [30]. A micro-SaaS solopreneur must personally handle support tickets, pricing, invoicing, sales-tax and VAT obligations, reporting and marketing. These tasks are repetitive and rule-driven yet still need judgement, so they consume the founder's time without being easy to automate with traditional if-this-then-that tools.

Why now. LLM agents can now plan, call real APIs and recover from errors [1][3][5], and the industry has standardised how agents connect to tools (MCP) and to each other (A2A) [24][25]. Large vendors have responded with agent marketplaces, but these target enterprises already on Salesforce or AWS [26][27], or consumers chatting inside ChatGPT [28]. The long tail of independent businesses has no trustworthy place to obtain agents that act on their own Stripe, Shopify, Gmail or help-desk accounts.

Why a marketplace and not a single product. Each business function needs deep domain knowledge (for example, tax rules across jurisdictions). Research shows that narrow, specialised agents outperform general ones [8][9]. A marketplace lets many independent developers each build one excellent agent while the platform supplies the hard, shared parts: identity, credential custody, sandboxing, certification, approvals, billing and monitoring. This is the classic two-sided platform opportunity [19].

Why trust is the core technical contribution. Delegating real business actions to third-party agents is risky. Agents remain unreliable across repeated trials [12], are vulnerable to indirect prompt injection from the very emails and tickets they process [13][14], and LLM app stores already contain misleading and malicious apps [16]. The project's novelty is therefore not just a catalogue but a **trust layer**: (1) mandatory certification with repeated-trial reliability (pass^k) and prompt-injection tests before listing; (2) least-privilege permission scopes and sandboxed execution; (3) policy limits and human-in-the-loop approval for irreversible actions; and (4) transparent reliability and cost metrics shown to buyers, countering the first-proposal and speed biases observed in agentic markets [18].

Expected impact. For solopreneurs: lower operating cost and hours reclaimed from routine operations, with the owner kept in control. For developers: a distribution channel and revenue for specialised agents. For research: an open, measurable testbed that answers questions the literature leaves open - how certification metrics correlate with production reliability, how much human approval is needed for acceptable risk, and which defences stop prompt injection in multi-tenant agent platforms.

Publication and funding potential. We target a derivative paper on "Certifying third-party LLM agents for multi-tenant marketplaces" (candidate venues: an IEEE or ACM software-engineering or AI-systems workshop, or a conference such as IEEE SOSE/ICSOC), reporting pass^k reliability, attack-success rates before and after our

defences, and cost per task across model providers. The working prototype, three certified reference agents and a usability study with real solopreneurs make the project suitable for presentation to potential funders at the SJSU Expo.

Chapter 3. Project Requirements

Requirements

Actors and scope

Table 2. Actors

Actor	Description	Main goals
Solopreneur (Buyer)	Owner-operator of a micro-SaaS or small online business; non-expert in AI	Find trustworthy agents, connect them to business accounts, keep control, see results and cost
Agent Developer (Seller)	Independent developer who builds a specialised agent	Publish, version and price agents; get paid; see usage and failures
Platform Administrator	Project team operating the marketplace	Certify agents, handle abuse reports, monitor platform health
External SaaS Systems	Stripe, Shopify, Gmail, Zendesk, QuickBooks (sandbox accounts)	Accessed by agents through MCP connectors with scoped OAuth tokens
LLM Providers	Anthropic, OpenAI, open-weight models	Reached only through the platform LLM gateway

Use cases

Table 3. Use-case catalogue

ID	Use case	Primary actor	Summary
UC-01	Register and sign in	All	Create account, verify email, sign in with SSO; choose Buyer and/or Developer role
UC-02	Publish agent	Developer	Upload agent package and manifest (capabilities, required connectors, permission scopes, pricing); submit for certification
UC-03	Certify agent	Administrator / pipeline	Automated static scan, sandbox functional tests, pass^k reliability and prompt-injection tests; approve or reject
UC-04	Discover agents	Solopreneur	Search by natural language or filters; compare reliability, cost and permissions
UC-05	Install and configure agent	Solopreneur	Grant scoped OAuth access to business systems, set policies (spend limits, approval rules, schedule)
UC-06	Run agent	System / Solopreneur	Trigger on schedule, webhook event or on demand; agent executes in sandbox
UC-07	Approve high-impact action	Solopreneur	Review a proposed action (e.g., refund > $100, price change > 10%) and approve, edit or reject
UC-08	Monitor activity	Solopreneur	View run history, actions taken, outcomes, LLM cost and errors
UC-09	Billing and payouts	Solopreneur / Developer	Subscribe or pay per use; developer receives revenue share
UC-10	Revoke / pause agent	Solopreneur	Immediately pause an agent and revoke its credentials
UC-11	Rate and report agent	Solopreneur	Leave rating or abuse report; triggers admin review
UC-12	Delegate to another agent	Agent	An installed agent discovers and invokes another certified agent through A2A, within the buyer's policies

Functional requirements (user stories)

Priorities follow the template: **E = Essential** (required for a working system at the end of 295B), **D = Desired** (planned and scheduled), **O = Optional** (stretch goals). Each requirement is uniquely identified, verifiable by the acceptance criterion given, and traceable to a use case, following the characteristics of a good SRS in IEEE 830 section 4.3.

Table 4. Functional requirements

ID	User story	Pri.	Trace	Acceptance criterion
FR-01	As a user, I want to register and sign in with email or Google SSO so that my account is secure.	E	UC-01	OIDC sign-in works; passwords never stored in plain text; sessions expire after 24 h
FR-02	As a user, I want role-based access (Buyer, Developer, Admin) so that each sees only permitted functions.	E	UC-01	Unauthorised API calls return HTTP 403; verified by automated tests for every endpoint
FR-03	As a developer, I want to scaffold an agent with the SDK/CLI so that I can start from a working template.	E	UC-02	agentmkt init creates a runnable agent and manifest; agentmkt run --local passes sample test
FR-04	As a developer, I want to publish an agent with a manifest declaring capabilities, connectors, scopes and price.	E	UC-02	Manifest validated against JSON Schema; invalid manifests rejected with field-level errors
FR-05	As a developer, I want semantic versioning of my agent so that buyers can stay on a stable version.	E	UC-02	New version does not affect existing installs until buyer upgrades; rollback to prior version possible
FR-06	As the platform, I want every submitted version certified before listing.	E	UC-03	Pipeline runs static scan, functional tests, pass^k (k=4) and injection suite; result and report stored
FR-07	As a solopreneur, I want to search agents in natural language and filter by category, price and rating.	E	UC-04	Query returns ranked results in < 1 s for 1,000 agents; filters combinable
FR-08	As a solopreneur, I want an agent detail page showing permissions, reliability score, cost estimate and reviews.	E	UC-04	All manifest scopes listed in plain language; pass^k and avg. cost/run shown
FR-09	As a solopreneur, I want to connect my business accounts via OAuth with only the scopes the agent needs.	E	UC-05	Tokens stored encrypted in vault; agent cannot call APIs outside granted scopes (tested)
FR-10	As a solopreneur, I want to set policies: spend caps, action limits, working hours and approval thresholds.	E	UC-05, UC-07	Policy violation blocks the action and notifies user; covered by unit tests
FR-11	As a solopreneur, I want agents to run on schedule, on webhook events or on demand.	E	UC-06	Cron, webhook and manual triggers each start a run; retried up to 3 times on transient failure
FR-12	As a solopreneur, I want to approve, edit or reject high-impact actions before they execute.	E	UC-07	Pending actions appear in console and by email; nothing executes without approval when rule applies
FR-13	As a solopreneur, I want a run history with every tool call, outcome and LLM cost.	E	UC-08	Each run shows timeline of steps; data retained 90 days; exportable as CSV
FR-14	As a solopreneur, I want to pause an agent and revoke its credentials instantly.	E	UC-10	Pause takes effect within 5 s; subsequent tool calls fail closed
FR-15	As the platform, I want an immutable audit log of every agent action and approval.	E	UC-06, UC-07	Append-only; records actor, agent version, inputs hash, outcome and timestamp
FR-16	As a team, we want three certified reference agents: customer support, dynamic pricing, invoicing/tax.	E	UC-06	Each passes certification and completes its scripted demo scenario end-to-end
FR-17	As a solopreneur, I want to subscribe or pay per use, and as a developer receive revenue share.	D	UC-09	Stripe test-mode subscriptions, metered usage and Connect payouts reconcile to the cent
FR-18	As a developer, I want usage and failure analytics for my agents.	D	UC-08	Dashboard shows installs, runs, success rate, top errors per version
FR-19	As a solopreneur, I want to rate and report agents.	D	UC-11	Ratings shown on detail page; report creates admin ticket
FR-20	As an agent, I want to delegate subtasks to another certified agent via A2A within the buyer's policies.	D	UC-12	Delegation requires buyer's prior consent; delegated calls appear in audit log
FR-21	As a solopreneur, I want a weekly digest of what agents did and money/time saved.	D	UC-08	Email digest generated every Monday with key metrics
FR-22	As a solopreneur, I want natural-language setup	O	UC-05	Generated policy shown for confirmation before

ID	User story	Pri.	Trace	Acceptance criterion
	("handle refunds under $50") converted into policies.			saving
FR-23	As a developer, I want a local simulator of buyer accounts for testing.	O	UC-02	Mock MCP servers for Stripe/Shopify/Zendesk available in SDK
FR-24	As a solopreneur, I want agents that pay for services on my behalf (ACP/AP2).	O	UC-06	Prototype payment through a test-mode agent payment protocol
FR-25	As a user, I want a mobile-friendly approval experience with push notifications.	O	UC-07	Approvals usable on 375 px wide screen; push via web notifications
FR-26	As a solopreneur, I want every agent offered in two delivery modes - ready-to-use (works immediately with platform-managed defaults) and configurable (I supply my own credentials and settings) - so that I can start fast or tailor the agent.	E	UC-05	Manifest declares its modes; console offers one-click install or a credentials and settings step; registry rejects manifests without a valid mode

Reference agent requirements

Table 5. Reference agents (Essential)

Agent	Behaviour	Connectors	Human approval required when
Customer-support agent	Classifies and answers tickets using the merchant's FAQ and order data; issues refunds and resends licence keys	Zendesk/Gmail, Stripe	Refund > configured limit; low-confidence answer; angry/escalation intent
Dynamic-pricing agent	Proposes price changes from sales velocity, competitor prices and sentiment within merchant bounds	Stripe/Shopify, web search	Any change > 10% or outside min/max bounds
Invoicing & tax agent	Generates invoices, computes sales tax/VAT from tax-rate service, flags filing deadlines	Stripe, QuickBooks, tax-rate API	Any filing or payment action (agent prepares, never files autonomously)

Non-functional requirements

Table 6. Non-functional requirements

ID	Category	Requirement (measurable)	Pri.
NFR-01	Security	All agent code runs in a sandbox with no network egress except the LLM gateway and the connectors in its manifest	E
NFR-02	Security	Credentials encrypted at rest (AES-256) and in transit (TLS 1.2+); never exposed to agent code or logs	E
NFR-03	Security	Prompt-injection attack success rate on our test suite below 5% for certified reference agents	E
NFR-04	Reliability	Reference agents achieve pass^4 >= 0.80 on their certification scenarios	E
NFR-05	Performance	Marketplace API p95 latency < 300 ms (excluding agent runs); search < 1 s	E
NFR-06	Scalability	Support 100 concurrent agent runs and 1,000 listed agents on the staging cluster	D
NFR-07	Availability	Staging environment 99% monthly uptime during 295B evaluation period	D
NFR-08	Cost	Average LLM cost per reference-agent run reported; target < $0.05 for support tickets	D
NFR-09	Usability	New solopreneur installs and configures an agent in < 10 minutes; SUS score >= 70 in user study	E
NFR-10	Accessibility	Web UIs meet WCAG 2.1 AA for colour contrast and keyboard navigation	D
NFR-11	Observability	100% of agent runs traced end-to-end (OpenTelemetry) with cost and token counts	E
NFR-12	Privacy	Tenant data isolated; buyer can delete account and all data within 24 h	E

ID	Category	Requirement (measurable)	Pri.
NFR-13	Maintainability	Back-end unit-test coverage >= 70%; CI runs on every pull request	E
NFR-14	Portability	Full stack starts locally with one docker compose up command	E
NFR-15	Compliance	Agents provide assistance only; tax/legal outputs carry a disclaimer and require human filing	E

Constraints and assumptions

- External systems are used in sandbox/test mode only (Stripe test mode, Shopify development stores, trial help-desk accounts); no real customer money moves during the project.

- LLM usage is funded through provider credits or student programmes; the LLM gateway enforces a monthly budget per environment.

- The marketplace is a research prototype; it is not offered to the public and does not provide legal or tax advice.

- The team has three members; scope is sized to three reference agents plus the platform.

Chapter 4. Dependencies and Deliverables

Dependencies

Only showstopper dependencies - those that would halt the project if unavailable - are listed, each with a mitigation.

Table 7. Showstopper dependencies (VK = Vineet Kumar, NG = Nikhil Hiro Ghind, SP = Sakshat Nandkumar Patil)

ID	Dependency	Why it is a showstopper	Mitigation	Owner
D-01	LLM API access and credits (Anthropic, OpenAI)	Agents cannot reason or act without a capable model	Apply for academic/startup credits; LiteLLM gateway allows switching to open-weight models (Llama via Ollama) locally	NG
D-02	Cloud hosting (AWS EKS/RDS or GCP equivalent) and credits	No shared staging environment for integration, demo and load tests	AWS Educate / GitHub Student credits; full stack also runs locally with Docker Compose and kind	VK
D-03	Sandbox accounts and OAuth apps for Stripe, Shopify, Gmail, Zendesk	Reference agents and connectors cannot be developed or demonstrated	Register developer/test accounts in S1; mock MCP servers as fallback	SP
D-04	Tax-rate data source (e.g., Stripe Tax test mode or a public rate API)	Invoicing & tax agent cannot compute correct taxes	Use Stripe Tax in test mode; fallback to static rate tables for selected jurisdictions	NG
D-05	Open-source agent stack: LangGraph, MCP SDK, A2A SDK, Temporal	Runtime and orchestration are built on these	Pin versions; wrap behind our SDK interfaces so one can be replaced	NG / VK
D-06	Sandboxing technology (gVisor; Firecracker for hardening)	Untrusted agent code cannot be run safely without isolation	Start with gVisor on Kubernetes; restricted containers with network policies as minimum	VK
D-07	Advisor approval of scope and evaluation plan	Work cannot be accepted for credit without it	Review requirements and schedule with Prof. Ranjan in early October	All
D-08	Solopreneur participants for usability study and IRB guidance if required	NFR-09 and research evaluation cannot be validated	Recruit from SJSU entrepreneurship community and indie-maker forums from S8; confirm IRB needs early	SP

Deliverables

Table 8. Project deliverables

#	Deliverable	Description
1	Marketplace web application	Developer Portal and Solopreneur Console (Next.js), deployed to cloud staging
2	Marketplace back-end services	Registry, catalog/search, certification, billing, orchestration services with REST APIs (OpenAPI spec)
3	Agent runtime and policy engine	Sandboxed execution plane with LLM gateway, guardrails and human-in-the-loop approvals
4	Agent Developer SDK and CLI	Python package to scaffold, test locally, package and publish agents; published on GitHub with docs
5	Three certified reference agents	Customer support, dynamic pricing, invoicing & tax
6	MCP connector set	Stripe, Shopify, Gmail, Zendesk, QuickBooks connectors with least-privilege scopes
7	Certification and evaluation harness	Automated pipeline with pass^k reliability and prompt-injection test suites; reusable benchmark dataset
8	Monitoring dashboards	Run history, cost and reliability dashboards for buyers; platform health dashboards for admins
9	Infrastructure as code	Terraform + Helm charts and GitHub Actions pipelines to recreate the environment

#	Deliverable	Description
10	Documentation	Architecture and API docs, developer guide, user guide, threat model
11	Evaluation report and dataset	Reliability, security, performance, cost and usability results
12	Derivative research paper	Paper on certifying third-party LLM agents for multi-tenant marketplaces, submitted to a peer-reviewed workshop or conference (subject to advisor agreement)
13	Demo video and Expo prototype	5-minute end-to-end demo and live booth demo

Chapter 5. Project Architecture

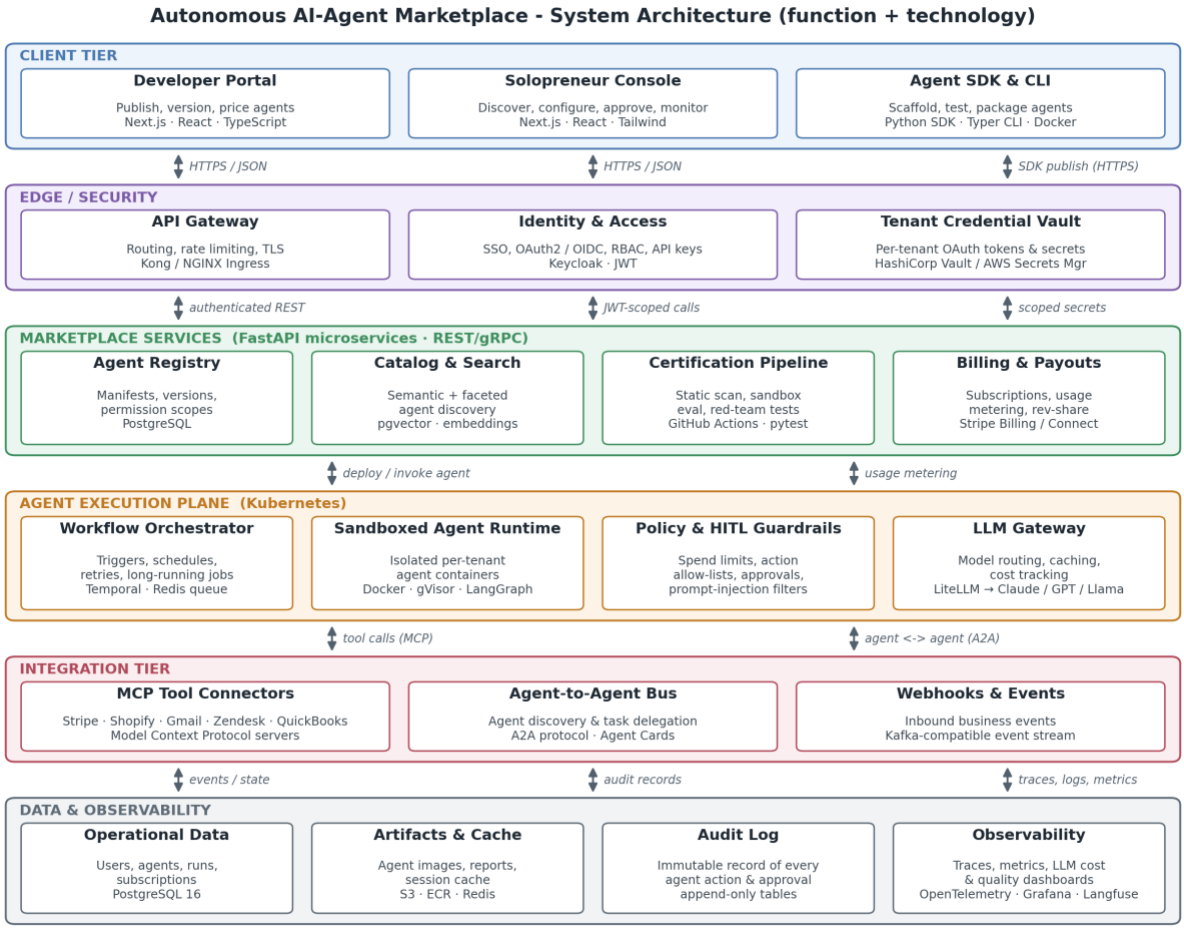


Figure 1. System architecture showing each component's function and technology

Architectural style. The system is a **layered, service-oriented (microservices) architecture with an event-driven execution plane**. Each tier has one responsibility and communicates with the next through authenticated REST/gRPC calls or events, so tiers can be developed by different team members and scaled independently [21].

Client tier. The *Developer Portal* lets developers register agents, upload versions, set prices and read certification reports and analytics. The *Solopreneur Console* is where buyers discover agents, connect accounts, set policies, approve pending actions and monitor runs. Both are Next.js/React/TypeScript apps. The *Agent SDK & CLI* (Python) scaffolds agents, runs them locally against mock connectors and publishes signed packages.

Edge and security tier. An *API gateway* (Kong or NGINX Ingress) terminates TLS, rate-limits and routes calls. *Identity & Access* (Keycloak, OAuth2/OIDC) issues JWTs carrying the user's role and tenant. The *Tenant Credential Vault* (HashiCorp Vault

or AWS Secrets Manager) holds each buyer's OAuth tokens; agents never see raw credentials - connectors receive short-lived, scope-restricted tokens at call time.

Marketplace services (Python FastAPI microservices over PostgreSQL). The *Agent Registry* stores manifests, versions and permission scopes. *Catalog & Search* combines keyword facets with semantic search over manifest embeddings (pgvector), ranking by certified reliability rather than recency to avoid the biases reported in [18]. The *Certification Pipeline* (GitHub Actions + pytest) runs static analysis, functional tests in the sandbox, repeated-trial pass^k tests [12] and prompt-injection tests [14]. *Billing & Payouts* uses Stripe Billing for subscriptions and metered usage and Stripe Connect for developer revenue share.

Agent execution plane (Kubernetes). The *Workflow Orchestrator* (Temporal) turns schedules, webhooks and manual triggers into durable runs with retries and timeouts. Each run executes in the *Sandboxed Agent Runtime*: a gVisor container per tenant and agent version, with network policies that allow egress only to the LLM gateway and approved connectors. Agents are built on LangGraph using a ReAct-style loop [3]. Every proposed tool call passes through the *Policy & HITL Guardrail* service, which checks scopes, spend limits and approval rules, filters suspected injected instructions, and pauses the workflow until the buyer approves when required. The *LLM Gateway* (LiteLLM) routes to Anthropic Claude, OpenAI GPT or open-weight models, caches prompts and records token cost per tenant.

Integration tier. *MCP tool connectors* wrap Stripe, Shopify, Gmail, Zendesk and QuickBooks as MCP servers [24] so any agent uses one uniform tool interface. The *Agent-to-Agent bus* implements A2A [25] so an agent can discover a certified agent through its Agent Card and delegate a task within the buyer's policies. *Webhooks & Events* ingest business events (new ticket, new order) into an event stream that triggers runs.

Data and observability tier. PostgreSQL holds users, agents, installations, runs and subscriptions; S3/ECR store agent images and generated artefacts; Redis caches sessions and queues. An append-only *audit log* records every action and approval. OpenTelemetry traces flow to Prometheus/Grafana for platform metrics and to Langfuse for LLM-level traces, cost and quality evaluation.

Key flow - running an agent. (1) A Zendesk webhook arrives at the gateway; (2) the orchestrator starts a run for the buyer's installed support agent; (3) the runtime loads the agent version in a sandbox and calls the LLM gateway; (4) the agent proposes a refund tool call; (5) the policy engine sees the amount exceeds the buyer's $50 limit and pauses the run; (6) the buyer approves in the Console; (7) the Stripe connector executes with a short-lived scoped token; (8) the outcome, cost and trace are written to the audit log and dashboard, and usage is metered for billing.

Deployment. Local development uses Docker Compose and kind (Kubernetes in Docker); staging runs on AWS EKS with RDS PostgreSQL, provisioned with Terraform and Helm and deployed by GitHub Actions.

Implementation update (Workbook 2). The structure of the architecture is unchanged. Implementation in Sprints 1-2 made four technology substitutions, explained in Chapter 6: a React 19 + Vite single-page console instead of Next.js; a PostgreSQL job queue with leases, retries and a cron scheduler instead of Temporal and Redis for CMPE 295A; hardened per-run containers first, with the gVisor runtime class added in Sprint 3; and our own LLM gateway, which adds model leases, admission control and local open-weight serving to the multi-provider routing planned with LiteLLM. The as-built status of every component is shown in Chapter 6.

Chapter 6. Project Design

This chapter documents the design as implemented by October 28, 2026 and as planned for the remaining sprints: the design methodology, the as-built components, UML class and sequence diagrams, the database design, the agent manifest, UI mockups and step-by-step instructions to reproduce the implementation.

Design Methodology

The team uses a **contract-first, walking-skeleton** methodology. Interfaces (the public OpenAPI specification, the SDK-to-broker API and the agent manifest JSON Schema) are versioned before the services behind them, so each member builds and tests a component against the contract and a simulator while other components are still in progress; a CI job fails when generated clients drift from the code. Each feature is delivered as a thin vertical slice (UI -> API -> database -> runtime -> agent) and hardened in later sprints. Four principles guide the design:

- **Brokers hold credentials.** Agents receive a short-lived, run-scoped capability token and never see OAuth tokens, API keys or the Docker socket; allowed actions are the intersection of the manifest request and the buyer's approval.

- **One system of record.** PostgreSQL with pgvector holds state, the job queue, run events, audit records and vectors.

- **Untrusted content is data.** Emails, tickets and documents are wrapped in evidence blocks with a random per-run boundary, and every side effect is decided in code from validated model output.

- **Human in the loop.** Actions above a buyer's policy threshold pause the run and create a durable approval request.

As-Built Component Design

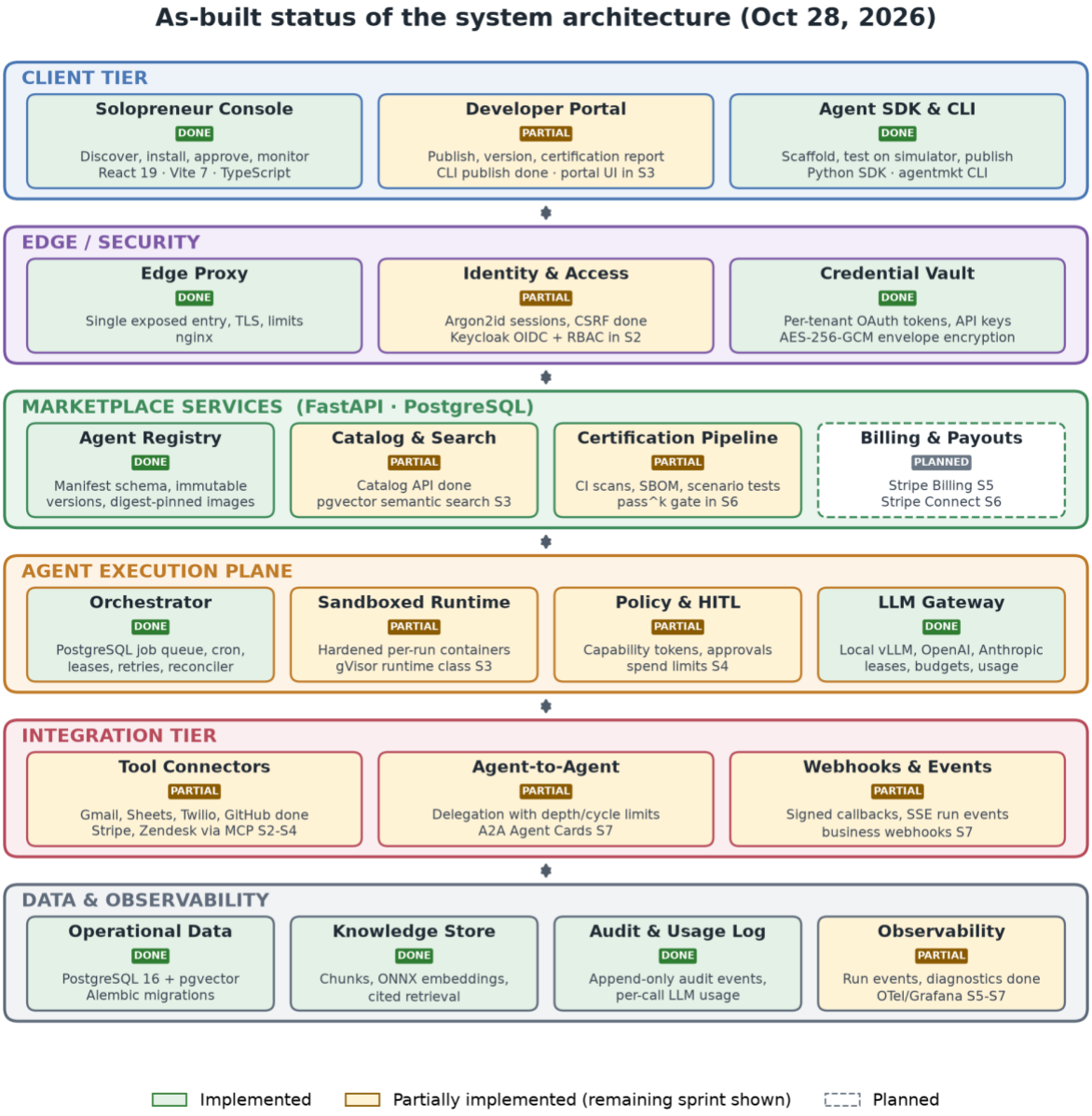


Figure 2. As-built status of each architecture component (October 28, 2026)

Ten of the twenty-one components are complete, ten are partially implemented and one (billing) is planned. Four technology choices changed during implementation: a React 19 + Vite single-page console instead of Next.js (no server rendering needed, smaller attack surface); a PostgreSQL job queue with leases, retries and a cron scheduler instead of Temporal and Redis; hardened per-run containers first, with gVisor added in Sprint 3; and our own LLM gateway, which adds model leases, admission control and local open-weight serving to multi-provider routing. Six agents built on the SDK exercise the design end to end: an email-triage agent (precursor of the customer-support agent), a

knowledge agent that writes cited briefs, a GitHub pull-request reviewer that delegates to a notifier agent, a telephony agent, a voice call-centre agent and a contract-probe agent used for acceptance tests.

Table 9. Component design: responsibility, implementation module, owner and status

Component	Responsibility	Module (repository)	Owner	Status
Solopreneur Console	Discover and install agents (ready-to-use or configurable), approve permissions and actions, follow runs, manage connections and schedules	apps/web (React 19, Vite 7, TypeScript)	SP	Done: 12 feature areas; two install flows in S2
Developer Portal	Publish versions, read certification reports and analytics	apps/web (developer routes)	SP	CLI publish done; portal UI in S3
Agent SDK and CLI	Typed run context (LLM, knowledge, input, events, connectors, idempotency, delegation); init, validate, test, build, publish; simulator	packages/python_sdk, packages/cli, packages/fake_platform	NG	Done
Control API (registry, catalog, runs)	Manifests and immutable versions, installations, runs, schedules, approvals, audit; owns the run state machine	services/control_api (FastAPI, SQLAlchemy, Alembic)	VK	Done; delivery-mode validation and multi-tenancy in S2
Scheduler / worker	Job queue with leases and retries, cron with misfire policy, reconciler	services/scheduler	VK	Done
Runtime daemon	Hardened per-run agent containers and model containers	services/runtime_daemon	NG	Done; gVisor in S3
Capability broker and vault	Checks every SDK call against the run token; holds connector credentials (AES-256-GCM envelope encryption)	services/capability_broker, packages/secret_store	SP	Done; spend limits in S4
LLM gateway	Local (vLLM), OpenAI and Anthropic routing; model leases; admission control; budgets; token usage	services/model_gateway	NG	Done
Knowledge and search	Chunking, local ONNX embeddings, pgvector retrieval with citations	services/knowledge	VK	Done; manifest search in S3
Connectors	Gmail, Sheets, Twilio and GitHub; Stripe and Zendesk as MCP tools	services/capability_broker	NG / SP	4 done; Stripe S2, Zendesk S3
Certification, billing	Scans, scenario tests, pass^k and injection gates; Stripe Billing and Connect	.github/workflows; services/billing	VK / SP	Scans done; gates S6; billing S5-S6

Domain Model and Key Interaction (UML)

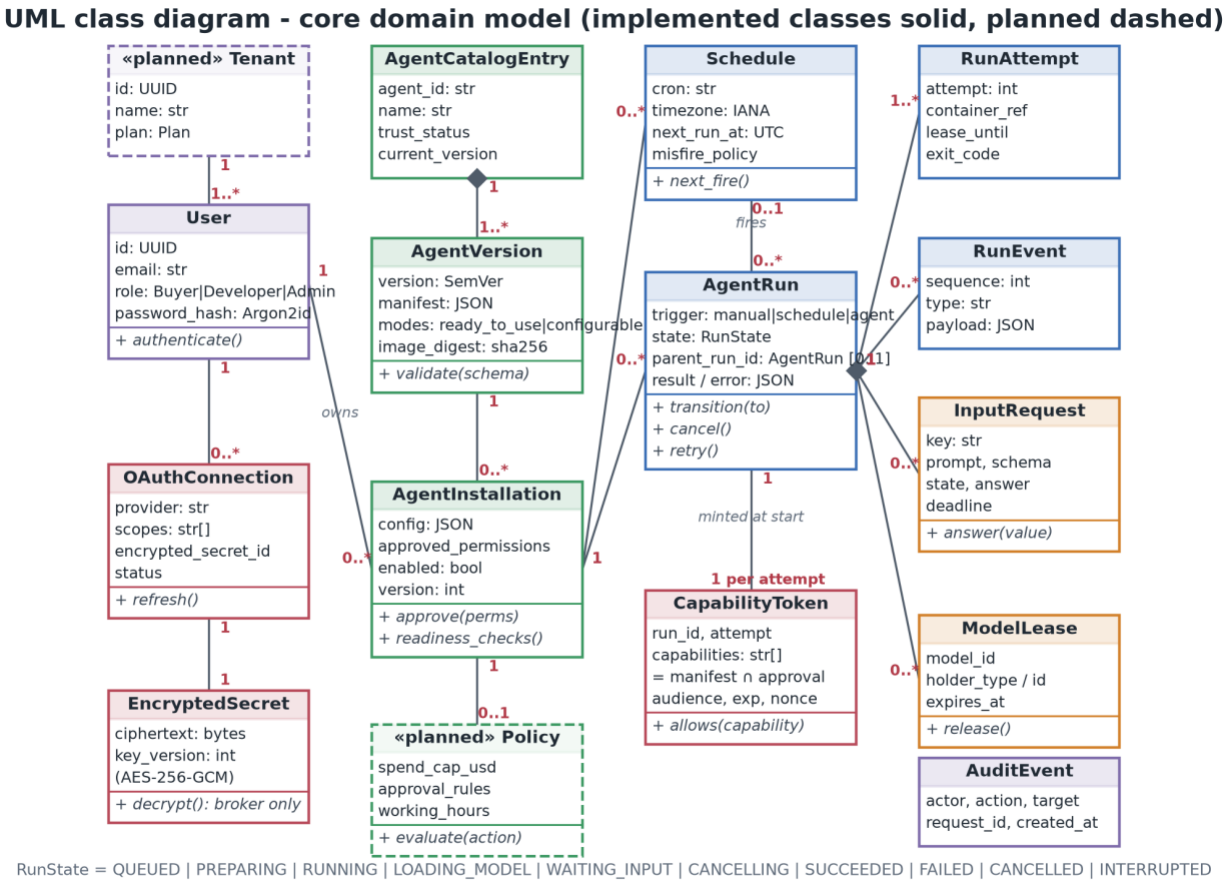


Figure 3. UML class diagram of the core domain model

An **AgentCatalogEntry** owns immutable **AgentVersion**s that store the manifest, the delivery modes and the image digest, so an installed agent never changes underneath a buyer (FR-05). An **AgentInstallation** binds a user to one version with the chosen mode, its configuration and the approved permissions. Each run attempt receives one **CapabilityToken** that the broker checks on every call. **Tenant** and **Policy** are added in Sprints 2 and 4.

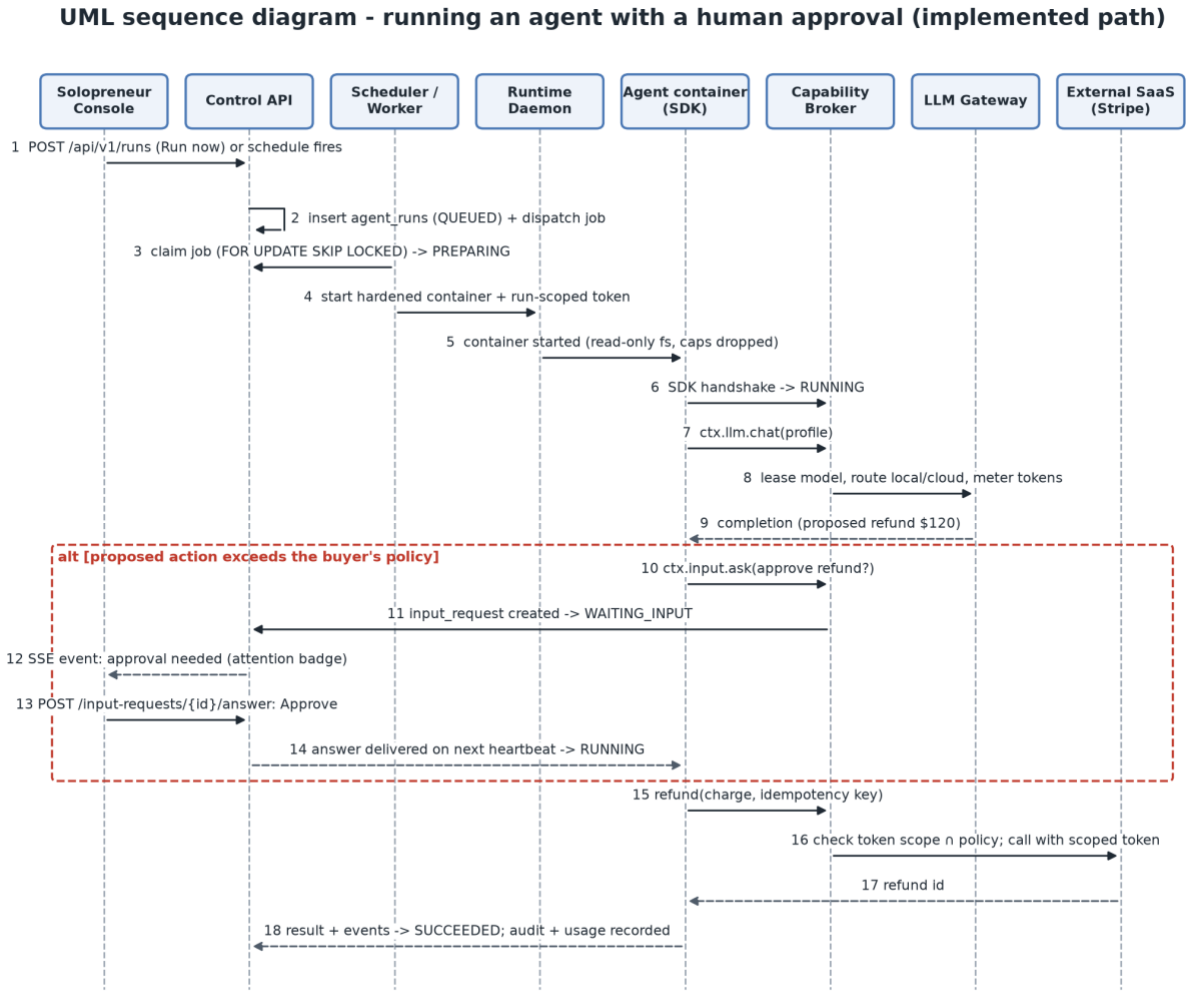


Figure 4. UML sequence diagram of an agent run that needs a human approval

The run row and its dispatch job are written in one transaction and a worker claims the job with FOR UPDATE SKIP LOCKED, so two workers never start the same run. When a proposed action exceeds the buyer's policy, the agent's ctx.input.ask call creates a durable approval request and the run waits in WAITING_INPUT; the external call is made only after approval and carries an idempotency key, so a retry cannot refund twice. Steps 1-14 are implemented; the Stripe refund tool (steps 15-17) arrives with the MCP adapter in Sprint 2.

Database Design

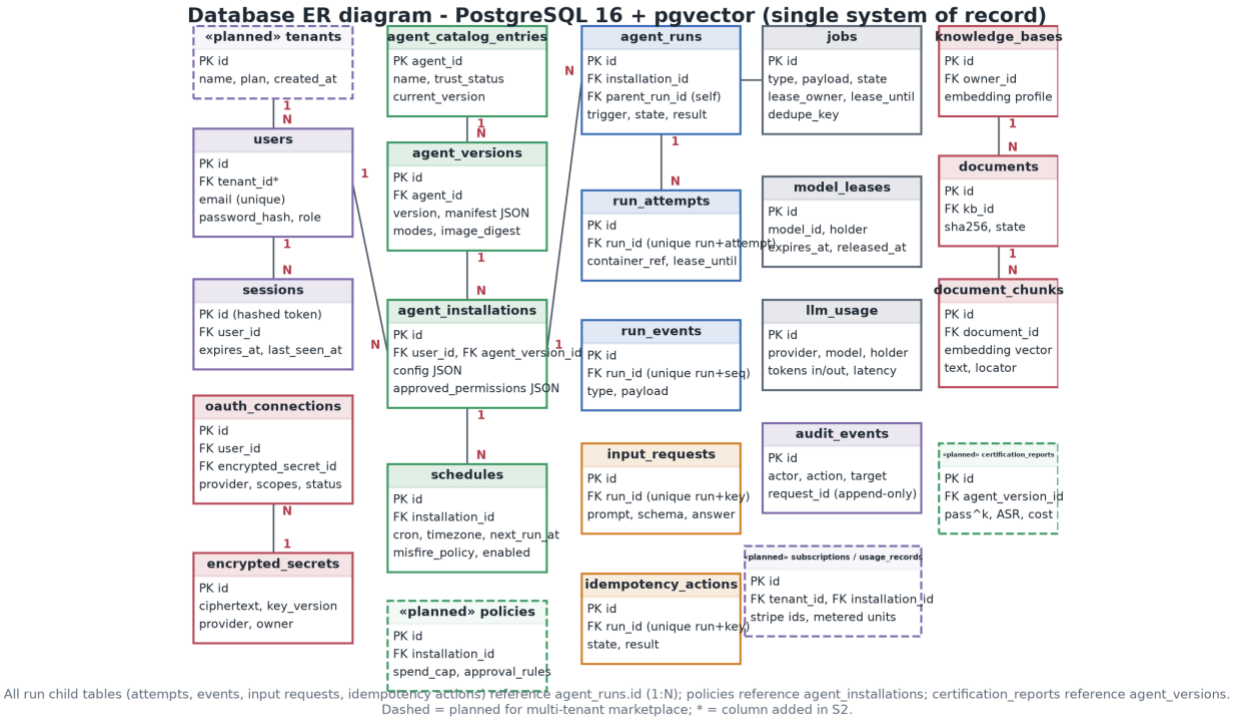


Figure 5. Entity-relationship diagram of the PostgreSQL database

All operational data lives in one PostgreSQL 16 database with pgvector: 29 tables managed by five Alembic migrations. Uniqueness constraints enforce correctness in the database itself: one attempt number per run, one sequence number per run event, one approval request per run and key, and one idempotency action per run and key.

Agent Manifest and Delivery Modes

Following the advisor meeting of September 23, every agent is offered in two delivery modes (FR-26): **ready-to-use**, which installs in one click with platform-managed defaults, and **configurable**, in which the buyer supplies their own credentials and settings. The manifest declares the supported modes and the configuration schema used by the configurable mode; the CLI (agentmkt validate) and the registry both validate it and reject invalid manifests with field-level errors (FR-04).

```
apiVersion: agentmkt/v1alpha1
kind: Agent
metadata: {id: support-refund-agent, version: 0.3.1}
spec:
  image: ghcr.io/agentmkt/support@sha256:<digest>
  modes: [ready_to_use, configurable]        # FR-26
  triggers: [manual, schedule, agent]
  permissions:
    llmProfiles: [local.general.small]
```

```
  connectors:
    zendesk: [tickets.read, tickets.write]
    stripe: [charges.read, refunds.write]
resources: {cpu: 2, memoryMb: 2048, activeTimeoutSeconds: 1800}
configurationSchema:                          # configurable mode
  required: [refundLimitUsd, zendeskSubdomain]
  properties:
    refundLimitUsd: {type: number, default: 50}
    zendeskSubdomain: {type: string}
```

UI Design

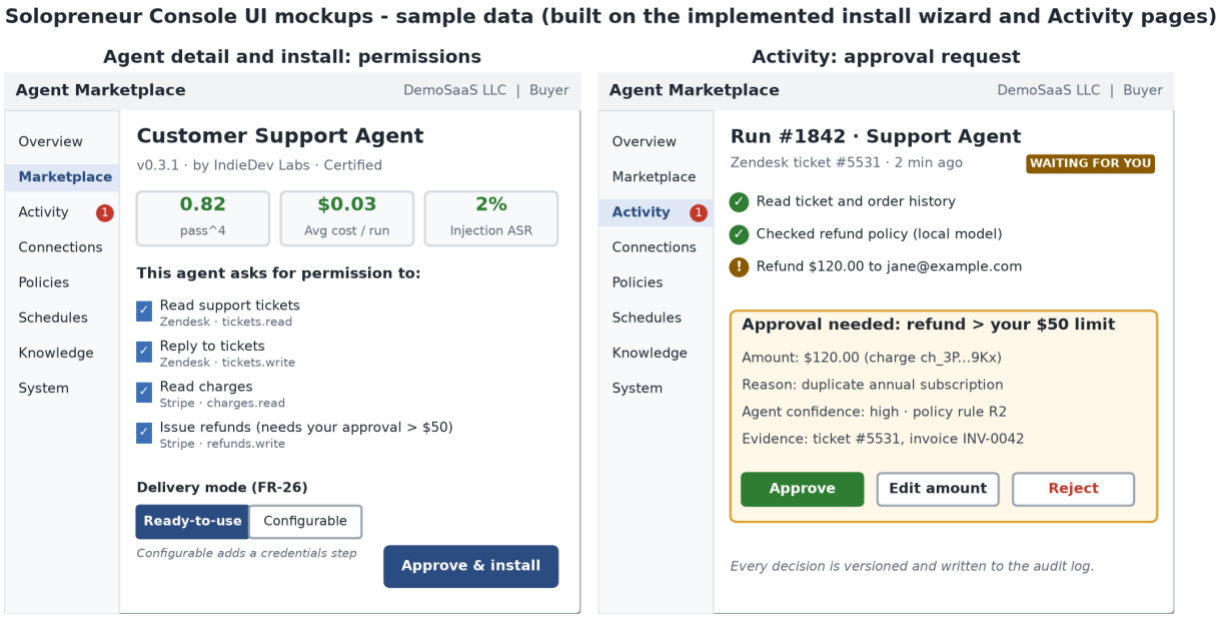


Figure 6. UI mockups of the agent install and approval screens (sample data)

The console already implements 12 feature areas (setup wizard, overview, marketplace, install wizard, activity and run timeline, approvals, connections, models, knowledge and chat, schedules, system and audit, backups). The mockups show the two screens extended for the marketplace: the install step lists every permission in plain language, the agent's certified metrics (FR-08) and the delivery-mode choice (FR-26); the approval card shows the evidence behind a proposed action (FR-12). The UI is tested with Playwright and axe against WCAG 2.1 AA (NFR-10).

Reproducing the Implementation

Prerequisites: macOS or Linux (amd64 or arm64), Docker with Compose v2, Python 3.12 with uv, Node 22.18 or later, git and make. No GPU or cloud account is needed; models are mocked unless an OpenAI or Anthropic key is added. CI runs the same commands on every pull request.

Step 1 - Clone and install

```
git clone \
https://github.com/vineetkumar-sjsu/CMPE_295A_Final_Project_FA26.git
cd CMPE_295A_Final_Project_FA26
make sync                    # Python workspace with uv
(cd apps/web && npm ci)      # web console
```

Step 2 - Start the platform and create the owner account

```
make demo-up            # full platform + demo agents on port 8080
make dev-bootstrap      # prints a one-time setup code
# then open http://localhost:8080/setup and create the account
```

Step 3 - Build, test and publish an agent

```
agentmkt init support-agent  # scaffold an agent and its manifest
cd support-agent
agentmkt validate            # JSON Schema and platform rules
agentmkt test                # scenarios against the simulator
agentmkt build --push        # build the image and pin its digest
agentmkt publish             # import the version into the catalog
```

Step 4 - Install and run from the console

Open Marketplace, choose the agent, pick a delivery mode and press Install; approve the permissions (and, for the configurable mode, enter credentials and settings), then press Run now. The Activity page shows the run live and questions from the agent appear as approval cards.

Step 5 - Verify

```
make db-up && make test-platform  # unit, integration, contract
make test-sdk                     # SDK, simulator, CLI, agents
make lint                         # ruff and mypy --strict
(cd apps/web && npm test && npx playwright test)
make perf                         # performance harness (Ch. 7)
make demo-down                    # stop; V=1 also deletes data
```

Table 10. Expected result of each step

Step	Expected result	If it fails
1	uv creates the virtual environment; npm ci completes	Install Python 3.12 with uv python install 3.12
2	demo-up returns when every health check passes (about 25 s from empty volumes); /setup accepts the code	Port 8080 busy: stop the other service; logs via make dev-logs
3	validate prints no errors; test reports every scenario passed; publish prints the version and digest	Docker must be running for build
4	The run reaches SUCCEEDED; container start is about 0.5 s on a laptop	Run stays in PREPARING: check make dev-logs
5	All suites pass; perf prints the target table of Chapter 7	test-platform needs PostgreSQL from make db-up

Chapter 7. QA, Performance, Deployment Plan

Quality Assurance Strategy

Testing follows a test pyramid with automated gates on every pull request. The repository contains 165 Python test modules with about 1,200 test functions, 11 web unit-test files and a Playwright end-to-end suite; continuous integration runs 17 jobs, and coverage thresholds are enforced on the security and state-machine modules.

Table 11. Test levels, tools and CI gates

Level	Scope	Tools	CI job	Owner
Static analysis	Lint, formatting, strict typing (Python); type-check and lint (TypeScript)	ruff, mypy --strict, tsc, eslint	lint, web	All
Unit and contract	State machine, queue, scheduler, capability checks, redaction, SDK; OpenAPI and client drift	pytest	platform-tests, contracts	All
Integration	Services against real PostgreSQL + pgvector; OAuth and telephony with fakes	pytest, Docker	platform-tests	VK / SP
Agent and real-stack E2E	Agents in hardened containers on the simulator (incl. injection scenarios) and through the real stack with failure injection	pytest harness	agent-e2e, realstack-e2e	NG / VK
Property / fuzz	Manifest parser and event redaction (three crash bugs found and fixed)	Hypothesis	fuzz	NG
UI and accessibility	Component tests, browser flows, WCAG checks	Vitest, Playwright, axe-core	web	SP
Security	Secret, dependency and image scanning with SBOMs (amd64, arm64)	gitleaks, scanners	secret-scan, image-security	SP
Performance and usability	Latency, dispatch, search at scale; SUS study with solopreneurs [33]	make perf; SUS	on demand; S9-S10	VK / SP

Evaluation Methodology

The evaluation answers six research questions. Each has a metric, an explicit baseline and an acceptance threshold from the requirements in Chapter 3.

Table 12. Research questions, metrics and baselines

Research question	Metric	Baseline	Status
RQ1. Is the platform fast enough for one interactive operator?	API p95; event visibility; run dispatch	Targets: NFR-05 (300 ms), FR-07 (1 s), events < 2 s, start < 10 s	Evaluated
RQ2. Does approximate vector search keep quality while cutting latency?	Query p95; recall@8 vs. exact	B1: exact k-nearest-neighbour search on the same chunks	Evaluated
RQ3. Are certified agents reliable across repeated trials?	pass^k (k = 4) [12]; cost per task	B2: pass@1 of the same agent; B3: rule-based triage and explore-exploit pricing [23]	Harness built; measured S5
RQ4. Do our defences stop indirect prompt injection?	Attack success rate (ASR) [14]	B4: undefended agent (untrusted text in prompt, tools exposed to model)	Defence in CI; measured S5/S8
RQ5. What does autonomy cost?	LLM cost and latency per run	B5: cloud model vs. local open-weight model	Metering done; measured S9
RQ6. Can a non-expert install and supervise an agent?	Install time (< 10 min); SUS (>= 70) [33]	B6: the same task done manually	S9-S10

Setup. A dedicated harness (make perf) starts its own Docker Compose project from empty volumes and sends every request through the edge proxy as a browser would. It exercises 13 API endpoints at concurrency 1, 16 and 64 (200-640 requests per endpoint per level), 10 sequential runs plus a burst of 12 concurrent runs of a real hardened agent container, and 20 questions x 5 rounds over 10,000 and 100,000 chunks embedded with a pinned local ONNX model (512 dimensions). Percentiles are nearest-rank. Hardware: Intel Core Ultra 7 255H, 30 GiB RAM, Docker 29.4, no GPU (model servers mocked), with other workloads running, so results are an upper bound.

Baseline B1: exact versus approximate vector search

Catalog search and agent knowledge retrieval use pgvector. The baseline is exact k-nearest-neighbour search (a parallel sequential scan); the treatment is an HNSW index [32] with pgvector defaults on the same table. Recall@8 is measured against the exact result for the same 100 queries.

Table 13. Baseline B1 results: exact search versus HNSW (p95 through the API, concurrency 1)

Chunks	Table size	Exact (baseline)	HNSW	Index build	HNSW recall@8
10,000	87 MB	83 ms (SQL 26 ms)	96 ms (planner kept the exact plan)	1.3 s	1.000
100,000	458 MB (719 MB with index)	426 ms (SQL 147 ms)	49 ms (SQL 7.4 ms)	10.0 s	0.994

Finding and decision. At 100,000 chunks HNSW is 8.7x faster at p95 (426 -> 49 ms) for a 0.6% loss in recall, but it adds 57% to table size; at 10,000 chunks PostgreSQL ignores the index. Exact search meets FR-07 but grows linearly (614 ms p95 at concurrency 4), so we keep exact search per knowledge base and add an HNSW index by migration for the shared catalog once it passes 50,000 rows.

Platform performance against requirement targets

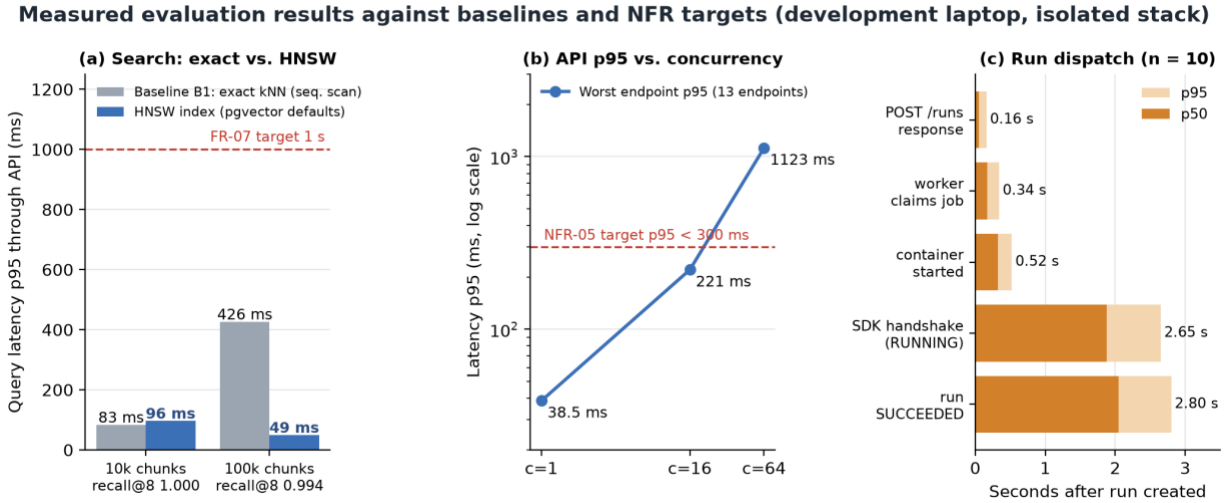


Figure 7. Measured baseline and performance results

Table 14. Measured results against requirement targets

Metric	Target	Measured	Result
Marketplace API p95, idle (c = 1), worst of 13 endpoints	< 300 ms (NFR-05)	38.5 ms	Pass
API p95 at c = 16 and c = 64	Headroom	221 ms and 1,123 ms; no errors	Pass at c = 16
Search p95 at 100,000 chunks	< 1 s (FR-07)	426 ms exact; 49 ms HNSW	Pass
UI event visibility after a server event	< 2 s	p95 510 ms (n = 690)	Pass
Agent container start, image present	< 10 s	p95 0.52 s; burst of 12: 7.9 s	Pass
Second model load while one is resident	Refused safely	20 of 20 refused (HTTP 409)	Pass
Fresh setup from empty volumes	One command (NFR-14)	27.4 s machine time	Pass

Analysis. All measured targets pass on a busy laptop. The tail at c = 64 is queueing in a single API process, so staging runs at least two API replicas and the Sprint 8 load test repeats the measurement at 100 concurrent runs (NFR-06). About 1.4 s of the time to RUNNING is Python start-up inside the agent image, which the SDK base image will reduce with pre-compiled, lazily imported modules.

Agent-level evaluation procedure (RQ3-RQ6)

- **Reliability.** At least 30 scenarios per reference agent (support tasks adapted from tau-bench [12]); each runs k = 4 times and pass^4 is the share passed in all four trials, compared with B2/B3 on the same scenarios. Target NFR-04: pass^4 >= 0.80.

- **Security.** AgentDojo-style injection cases [14] embedded in tickets, emails and documents; ASR is the share of cases with an action not justified by the task, compared with the undefended baseline B4. Target NFR-03: ASR < 5%.

- **Cost and usability.** Cost per run from the gateway's usage records across local and cloud models (B5; NFR-08 < $0.05 per ticket); 8-10 solopreneurs install an agent and handle one approval (B6; NFR-09 < 10 min, SUS >= 70). Means are reported with 95% bootstrap confidence intervals and McNemar's test for paired pass/fail outcomes.

Deployment Plan

Four environments are used: **local development** (make dev-up or make demo-up, Docker Compose; done), **integration** (real agent and model containers through the runtime daemon, run in CI; done), **performance** (isolated make perf stack; done) and **staging on AWS** (EKS, RDS PostgreSQL, S3/ECR provisioned with Terraform and Helm; Sprint 5), which is promoted with a release tag for the Expo. Every pull request runs the 17 CI jobs; merges to main build multi-architecture images pinned by digest and, from Sprint 5, deploy to staging. Database changes ship as forward-only Alembic migrations with a backup taken first by the implemented backup command, and rollback redeploys the previous image digests; agent versions are immutable, so buyers stay on the version they approved.

Chapter 8. Implementation Plan and Progress

Implementation follows a **walking-skeleton** strategy: first make the thinnest end-to-end path work (publish an agent -> discover it -> install it -> run it in a sandbox -> see the result), then deepen each component in later sprints. Status values below reflect the state of the repository on the date of this workbook; the progress log is updated at the end of every sprint.

Status at Workbook 2 (October 28, 2026). Sprint 1 is complete and Sprint 2 is in its second week. The project is ahead of schedule: Sprint 1 delivered its environment, CI and walking skeleton, and twelve work items that Workbook 1 scheduled for Sprints 2 to 10 are already done (listed in the table Work completed ahead of the Workbook 1 plan). The local end-to-end vertical slice - publish, discover, install, run in a sandbox, approve and see the result - which was the content of milestone M2 on December 11, already runs on a laptop. Two Sprint 1 spikes were re-sequenced: Keycloak moved into Sprint 2 together with RBAC, and gVisor into Sprint 3 after the hardened container runtime. Changes since Workbook 1 are shown in bold in this chapter and in Chapter 9.

Programming and Execution Environment

Table 15. Programming and execution environment

Layer	Technology (version)	Purpose	Owner	Status
Source control & CI	GitHub monorepo, GitHub Actions	Code hosting, pull-request reviews, CI on every PR	VK	**Done: monorepo, 17 CI jobs on every pull request**
Back-end services	Python 3.12, FastAPI, SQLAlchemy, Pydantic v2	Marketplace microservices and APIs	VK	**Done (S1); multi-tenant in S2**
Agent runtime	**Own Python SDK + capability broker + LLM gateway (local vLLM, OpenAI, Anthropic); MCP adapter in S2**	Agent loop, tool calls, delegation, model routing	NG	**Done (S1-S2)**
Front-end	**Node 22, React 19, Vite 7, TypeScript 5.9, TanStack Query (replaces Next.js)**	Developer Portal and Solopreneur Console	SP	**Done: 12 feature areas**
Data	**PostgreSQL 16 + pgvector; Redis not needed**	Operational data, semantic search, cache/queue	VK	**Done: 29 tables, 5 migrations**
Orchestration	**PostgreSQL job queue + cron scheduler (Temporal deferred)**	Durable, retryable agent workflows	VK	**Done in S1 (planned S4)**
Identity & secrets	**Local auth (Argon2id, CSRF) and AES-256-GCM secret store; Keycloak OIDC**	OIDC sign-in, RBAC, credential custody	SP	**Secrets done; OIDC in S2**
Containers	**Docker, Docker Compose, hardened per-run containers; gVisor**	Local stack and sandboxed agent execution	VK / NG	**Compose and hardened runtime done; gVisor S3**
Cloud	AWS EKS, RDS, S3, ECR; Terraform, Helm	Shared staging environment	VK	S5
Observability	OpenTelemetry, Prometheus, Grafana, Langfuse	Traces, metrics, LLM cost	SP	**Run events, token usage, diagnostics done; OTel S5**

Development Tools per Team Member

Table 16. Development tools acquired per member (target: all installed by end of Sprint 1, Oct 18)

Tool	Nikhil Hiro Ghind	Vineet Kumar	Sakshat N. Patil
IDE: VS Code / Cursor with Python, TS, Docker extensions	Yes	Yes	Yes
Docker Desktop, kind, kubectl, Helm	Yes	Yes	Yes
Python 3.12 + uv, Poetry; pytest	Yes	Yes	Yes
Node 22.18 + npm; Vitest, Playwright, axe-core	-	-	Yes
LLM API keys (Anthropic, OpenAI) via shared team gateway	Yes	Yes	Yes
Ollama with an open-weight model for offline testing	Yes	-	-
AWS account (IAM user), Terraform CLI	-	Yes	Read-only
Stripe CLI + test account; Shopify Partner dev store	Yes	-	Yes
Postman / Bruno for API testing	Yes	Yes	Yes
Figma (UI mockups)	-	-	Yes
Project tracking: GitHub Projects board	Yes	Yes	Yes
uv workspace, ruff, mypy --strict, make, Hypothesis (fuzz)	**Yes**	**Yes**	**Yes**

Prototyping and Simulation Environment

The project is software-only, so the "hardware prototyping" environment is a reproducible simulated business:

- **Local cluster:** Docker Compose for services plus a kind cluster with gVisor runtime class to exercise sandboxing and network policies on a laptop. **Implemented with Docker Compose (make dev-up, make integration-up); the kind cluster with gVisor follows in S3.**

- **Simulated solopreneur business ("DemoSaaS")**: a Stripe test-mode account with products, subscriptions and ~500 synthetic customers; a Shopify development store; a Zendesk trial seeded with ~300 synthetic support tickets generated from public FAQ-style templates. **Google, Twilio and GitHub test accounts connected; Stripe test mode and Zendesk trial in S2-S3.**

- **Mock MCP servers** that replay recorded API responses, so tests are deterministic and cost-free. **Implemented as the agent simulator (fake broker with mock Gmail, Sheets, Twilio, LLM and knowledge).**

- **Evaluation sandbox** adapting τ-bench-style retail scenarios [12] and AgentDojo-style injection cases [14] to our connectors. **Scenario harness with scripted models and injection scenarios implemented; tau-bench adaptation in S5.**

- **Cloud staging** on AWS EKS for integration, load tests (k6/Locust) and the demo.

- **Isolated performance stack (make perf) that measures the platform from empty volumes without touching other stacks.**

- **Real-stack harness that runs real agent images through the real broker, with capability-matrix, isolation and failure-injection tests.**

Sample Programs and Technology Spikes

Table 17. Sample programs used to learn each technology

Spike	Goal / what it proves	Owner	Target
SP-1 Hello agent	LangGraph ReAct agent calls a local MCP server (Stripe test mode: list customers) through LiteLLM	NG	Oct 18 **- Done**
SP-2 Sandboxed run	Run SP-1 inside a gVisor container on kind with egress limited by NetworkPolicy	NG / VK	Oct 18 **- Done (gVisor S3)**
SP-3 Registry API	FastAPI service stores and validates an agent manifest (JSON Schema) in PostgreSQL	VK	Oct 18 **- Done**
SP-4 Durable workflow	Temporal workflow that pauses for a human approval signal and resumes	VK	Oct 25 **- Done (PostgreSQL queue)**
SP-5 Auth + console	Next.js app signs in via Keycloak OIDC and calls a protected API	SP	Oct 18 **- Partial (OIDC S2)**
SP-6 OAuth connect	Console completes Stripe/Shopify OAuth and stores token in Vault	SP	Oct 25 **- Partial (Stripe S2)**
SP-7 A2A call	One agent discovers another's Agent Card and delegates a task	NG	Nov 1 **- Partial (A2A S7)**

Draft agent manifest used by SP-3 (simplified):

```
name: support-refund-agent
version: 0.1.0
capabilities: [answer_ticket, issue_refund]
connectors:
  - zendesk: [tickets.read, tickets.write]
  - stripe:  [charges.read, refunds.write]
policies_required: [refund_limit_usd, approval_channel]
pricing: {model: per_run, usd: 0.02}
runtime: {image: ghcr.io/agentmkt/support:0.1.0, timeout_s: 120}
```

The draft manifest above is superseded by the validated v1alpha1 manifest in Chapter 6 (Agent Manifest and Delivery Modes), which adds the two delivery modes.

Prototype Implementation Plan

Table 18. Prototype increments (end of CMPE 295A)

Step	Increment	Technologies exercised	Sprint
1	Walking skeleton: monorepo, Compose stack, CI, health-	GitHub Actions, Docker, FastAPI,	S1 **- Done**

Step	Increment	Technologies exercised	Sprint
	check endpoints, empty UIs	Next.js	
2	Publish path: SDK -> manifest validation -> registry -> portal listing	SDK/CLI, Pydantic, PostgreSQL	S2 **- Done (CLI -> catalog); portal UI S3**
3	Discover & install: semantic search, detail page, OAuth connect, policy form	pgvector, Keycloak, Vault	S3-S4 **- Install done; search S3, OIDC S2**
4	Run path: trigger -> Temporal workflow -> sandboxed agent -> MCP tools -> result	Temporal, gVisor, LangGraph, MCP, LiteLLM	S3-S4 **- Done (PostgreSQL queue instead of Temporal)**
5	Approval path: policy engine pauses run; buyer approves in console	Temporal signals, WebSockets	S4 (basic) / S6 **- Basic done; policy-triggered S4**
6	Visibility: run timeline, audit log, cost per run	OpenTelemetry, Langfuse	S5 **- Done except OTel (S5)**
7	Deploy prototype to EKS; demo customer-support agent end-to-end (Milestone M2)	Terraform, Helm, EKS	S5 **- S5; local demo already runs**
8	**Multi-tenant data model; Keycloak OIDC and RBAC**	**PostgreSQL, Keycloak**	**S2**
9	**MCP tool adapter; Stripe and Zendesk connectors**	**MCP Python SDK, Stripe test mode**	**S2-S3**
10	**Evaluation harness v1: pass^k and injection baseline**	**pytest, simulator, LLM gateway**	**S5**
11	**Stripe Billing usage metering**	**Stripe Billing, llm_usage**	**S5**

Work Completed Ahead of Plan

The table compares items finished by October 28 with the sprint in which Workbook 1 scheduled them. The repository now holds about 77,000 lines of Python (excluding tests) and 18,000 lines of TypeScript, 29 database tables, 67 public API paths, six working agents and about 1,200 automated tests.

Table 19. Work completed ahead of the Workbook 1 plan

Work item	Planned (WB1)	Completed	Sprints ahead	Owner
Agent manifest schema and Registry/Catalog API	**S2**	**S1**	**1**	**VK**
Python SDK, CLI and local runner	**S2**	**S1**	**1**	**NG**
Capability broker and encrypted credential store (SP-6)	**S2**	**S1**	**1**	**SP**
Sandboxed per-run runtime (hardened containers)	**S3**	**S1**	**2**	**NG**
Job queue, cron scheduler and run state machine (SP-4)	**S4**	**S1**	**3**	**VK**
Install wizard, permission approval and configuration forms	**S4**	**S2**	**2**	**SP**
Run history, activity timeline and audit view	**S5**	**S2**	**3**	**SP**
Basic human-in-the-loop approvals (input requests, notifications)	**S6**	**S2**	**4**	**NG**
Agent-to-agent delegation prototype	**S8**	**S2**	**6**	**VK**
CI security scans: SBOM, image, secret and dependency scanning	**S9**	**S2**	**7**	**SP**
Performance harness and baseline evaluation	**S10**	**S2**	**8**	**VK**
Agent simulator with mock connectors (FR-23, Optional)	**Not scheduled**	**S1**	**-**	**NG**

Progress Log

Table 20. Implementation progress (updated each sprint)

Date	Item	Status
Sep 1, 2026	Project repository created; README with concept, goals, team and roadmap	Done
Sep 2026	Project concept and abstract agreed with advisor	Done
Sep 2026	Literature search, state-of-the-art review, requirements, architecture (this workbook)	Done
Oct 5-18, 2026	Sprint 1: environment, tools, spikes SP-1, SP-2, SP-3, SP-5	**Done (OIDC part of SP-5 moved to S2)**
Oct 19-Nov 1, 2026	Sprint 2: publish path; spikes SP-4, SP-6, SP-7	**In progress (SP-4 done; SP-6, SP-7 partial)**
Sep 23, 2026	**Advisor meeting: two delivery modes (ready-to-use, configurable) adopted as a core requirement (FR-26)**	**Done**
Oct 5-18, 2026	**Monorepo, Docker Compose stack, PostgreSQL 16 + pgvector, Alembic migrations, 17 CI jobs; Control API (registry, catalog, runs, schedules, audit); scheduler and hardened runtime**	**Done**
Oct 5-18, 2026	**Python SDK, CLI and simulator; capability broker with run-scoped tokens and AES-256-GCM vault; manifest mode field and configuration schema (NG)**	**Done**
Oct 19-28, 2026	**Web console (marketplace, install wizard, Activity, approvals, connections, schedules); LLM gateway; knowledge service; agent delegation**	**Done**
Oct 19-28, 2026	**Injection defences and scenarios, fuzz suite, image and secret scans; performance harness and baseline evaluation (Chapter 7)**	**Done**
Oct 19-Nov 1, 2026	**Registry stores and validates the agent mode (VK); one-click and configurable install flows (SP); multi-tenancy, OIDC + RBAC, MCP + Stripe**	**In progress**
Oct 28, 2026	**Workbook 2: design, QA and evaluation, updated plan and schedule**	**Done**

Chapter 9. Project Schedule

The project uses two-week Scrum sprints across CMPE 295A (Fall 2026) and CMPE 295B (Spring 2027), tracked on a GitHub Projects board. Each sprint ends with a demo to the team and a progress-log update. Every member owns one stream of work in every sprint, so tasks do not overlap; integration is done jointly at the end of each sprint. **For Workbook 2 the remaining sprints were re-planned: work finished early freed capacity, so items from Sprints 6-10 were pulled forward by one sprint each, Sprint 10 now holds optional features and a schedule buffer, and milestone M3 moves two weeks earlier. The Gantt chart covers both semesters, and the PERT charts that follow show the task dependencies and the critical path.**

Table 21. Milestones

Milestone	Date	Exit criteria
M1 Architecture baseline	Oct 4, 2026 - **achieved**	Requirements, architecture and schedule reviewed with advisor
M2 End-to-end prototype	Dec 11, 2026 - **on track; local slice done Oct 28**	Customer-support agent published, discovered, installed and run in sandbox on cloud staging
M3 Beta	**Feb 21, 2027 (was Mar 7)**	Three certified reference agents, approvals, billing and dashboards working
M4 Final release	May 7, 2027 **- on track**	Hardened, evaluated system; paper draft; demo video; Expo ready

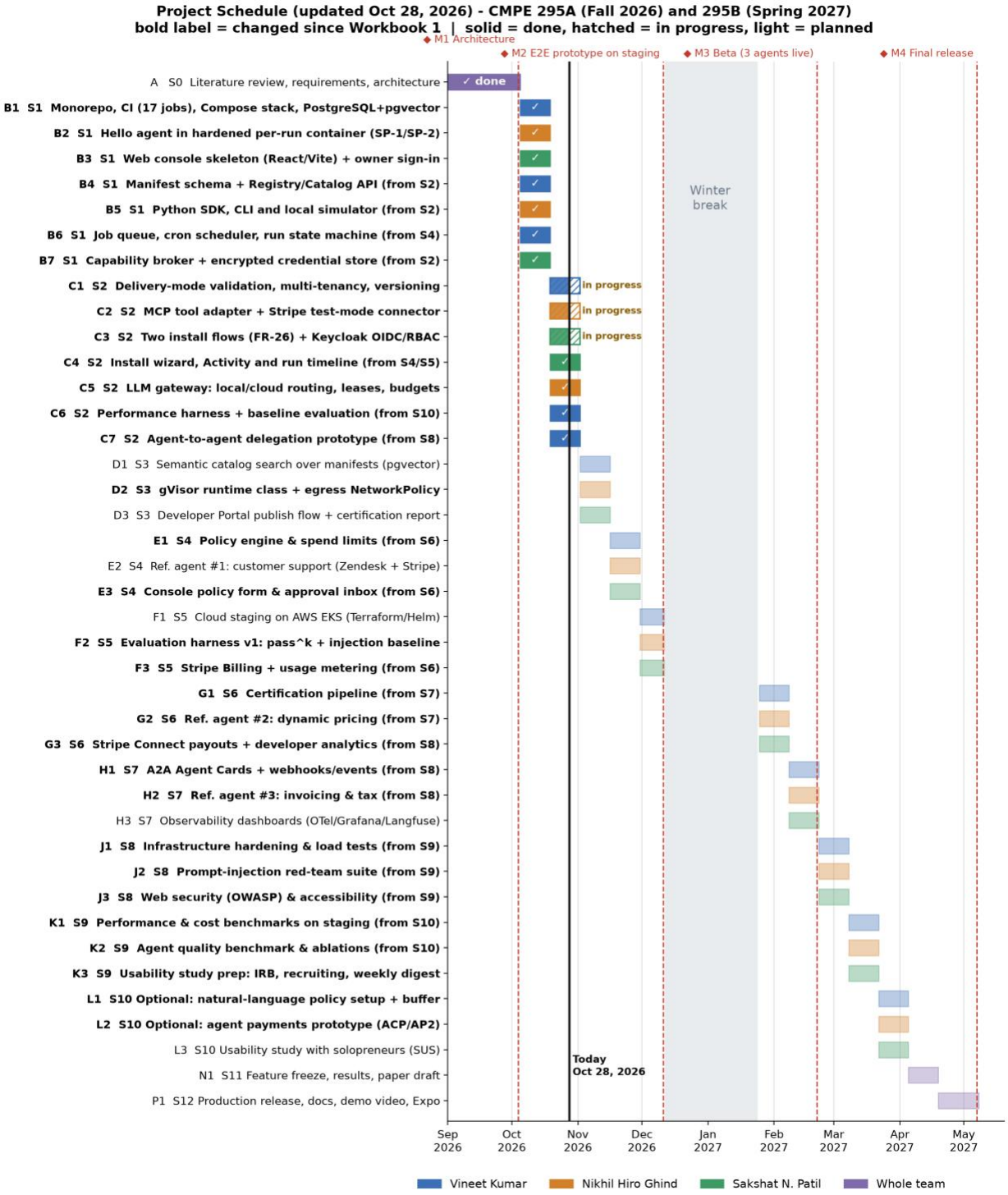


Figure 8. Project schedule (Gantt chart) by sprint and team member - updated with status and re-plan for both semesters

Table 22. Task assignment per sprint (no overlapping tasks between members)

Sprint (dates)	Vineet Kumar	Nikhil Hiro Ghind	Sakshat N. Patil
S0 (Sep 1-Oct 4) - done	Systems & platform literature; architecture	Agent literature; agent requirements	Security literature; state of the art; requirements
S1 (Oct 5-18)	**Monorepo, CI/CD, Compose stack, Postgres + registry API and job queue/scheduler (done)**	**Hello agent on hardened container; SDK, CLI, simulator (done); gVisor moved to S3**	**Wireframes; React/Vite console; owner sign-in; capability broker + credential store (done)**
S2 (Oct 19-Nov 1)	**Delivery-mode validation in registry; multi-tenant data model; versioning; perf harness and delegation (done)**	**MCP tool adapter + Stripe connector; LLM gateway (done)**	**Ready-to-use and configurable install flows; Keycloak OIDC + RBAC; install wizard and Activity (done)**
S3 (Nov 2-15)	Catalog & semantic search	**gVisor runtime class + egress NetworkPolicy**	**Developer Portal publish flow + certification report**
S4 (Nov 16-29)	**Policy engine & spend limits (from S6)**	Reference agent #1: customer support	**Policy form & approval inbox (install/configure done in S2)**
S5 (Nov 30-Dec 11)	EKS staging (Terraform/Helm)	**Evaluation harness v1: pass^k + injection baseline**	**Stripe Billing + usage metering (from S6); run history done in S2**
Winter break	Design docs for Workbook 2 (optional)	Threat model (optional)	UI mockups v2 (optional)
S6 (Jan 25-Feb 7)	**Certification pipeline (from S7)**	**Reference agent #2: dynamic pricing (from S7)**	**Stripe Connect payouts + developer analytics (from S8)**
S7 (Feb 8-21)	**A2A Agent Cards + webhooks/events (from S8)**	**Reference agent #3: invoicing & tax (from S8)**	Observability dashboards
S8 (Feb 22-Mar 7)	**Infrastructure hardening & load tests (from S9)**	**Prompt-injection red-team suite (from S9)**	**Web security (OWASP) & accessibility (from S9)**
S9 (Mar 8-21)	**Performance & cost benchmarks on staging (from S10)**	**Agent quality benchmark & ablations (from S10)**	**Usability study prep: IRB, recruiting; weekly digest**
S10 (Mar 22-Apr 4)	**Optional: natural-language policy setup; schedule buffer**	**Optional: agent payments prototype (ACP/AP2)**	Usability study with solopreneurs
S11 (Apr 5-18)	Results: performance section of paper	Results: reliability/security sections	Results: usability section; figures
S12 (Apr 19-May 7)	Production release & runbooks	SDK docs & sample agents	User guide, demo video, Expo booth

PERT Chart and Task Dependencies

The PERT charts lay out every work package of both semesters as an activity-on-node network. Each node shows the owner, the expected duration te = (O + 4M + P) / 6 from optimistic, most-likely and pessimistic estimates in working days, the earliest start-finish window and the slack from a critical-path calculation; arrows are finish-to-start dependencies. The critical path (red) runs A -> B1/B2/B3 -> B5/B7 -> C2 (MCP adapter and Stripe connector) -> E2 (customer-support agent) -> F2 (evaluation harness) -> G1 (certification) -> H1 (A2A and webhooks) -> J1 (hardening) -> K1 (benchmarks) -> N1 (paper) -> P1 (release). Its expected length is 130 working days against about 150 available between September 1 and May 7 (winter break excluded), leaving about four weeks of float. Task IDs match the bars of the Gantt chart, and the task-assignment table above states what each member does in every sprint.

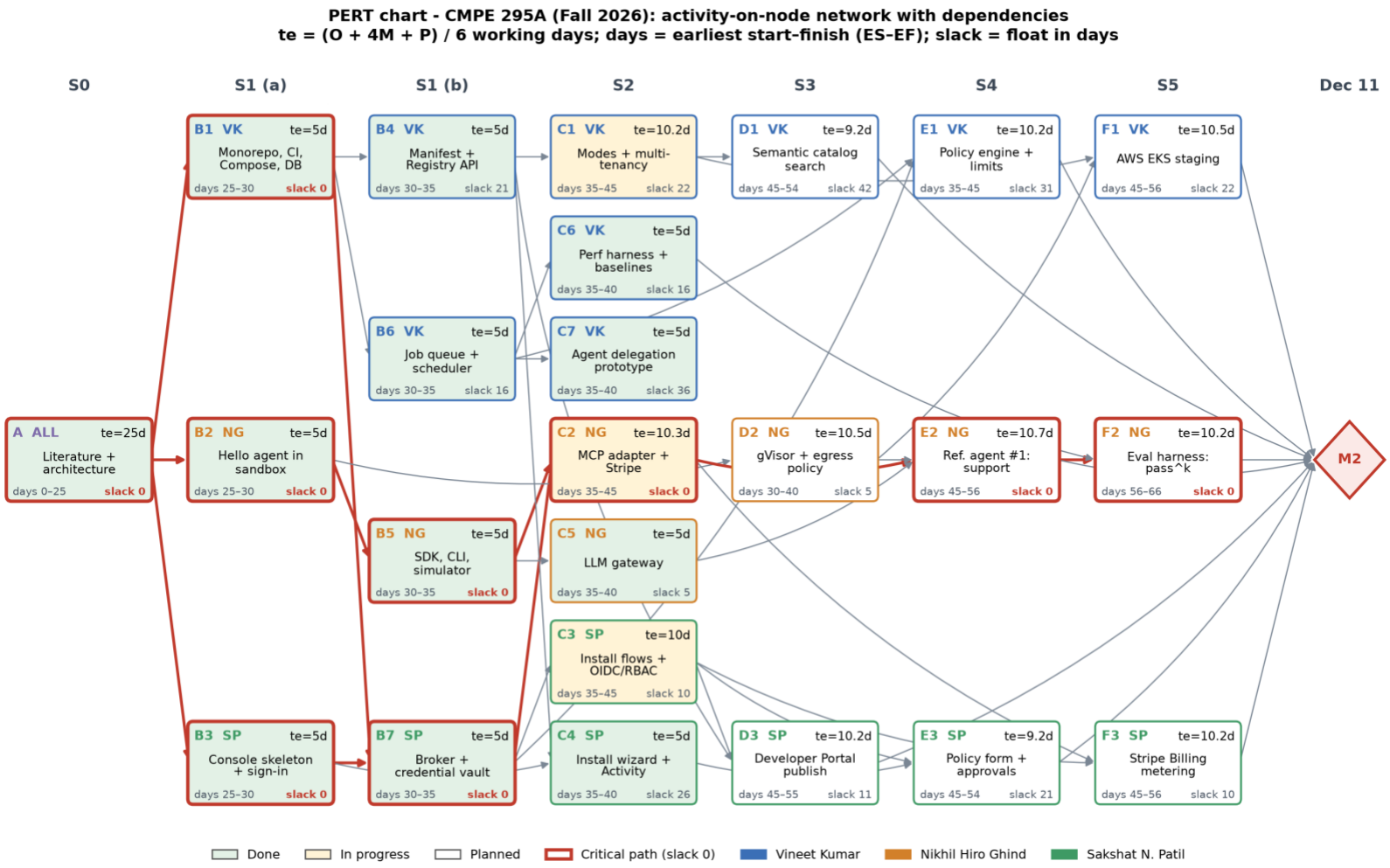


Figure 9. PERT chart for CMPE 295A (Fall 2026) with dependencies and critical path

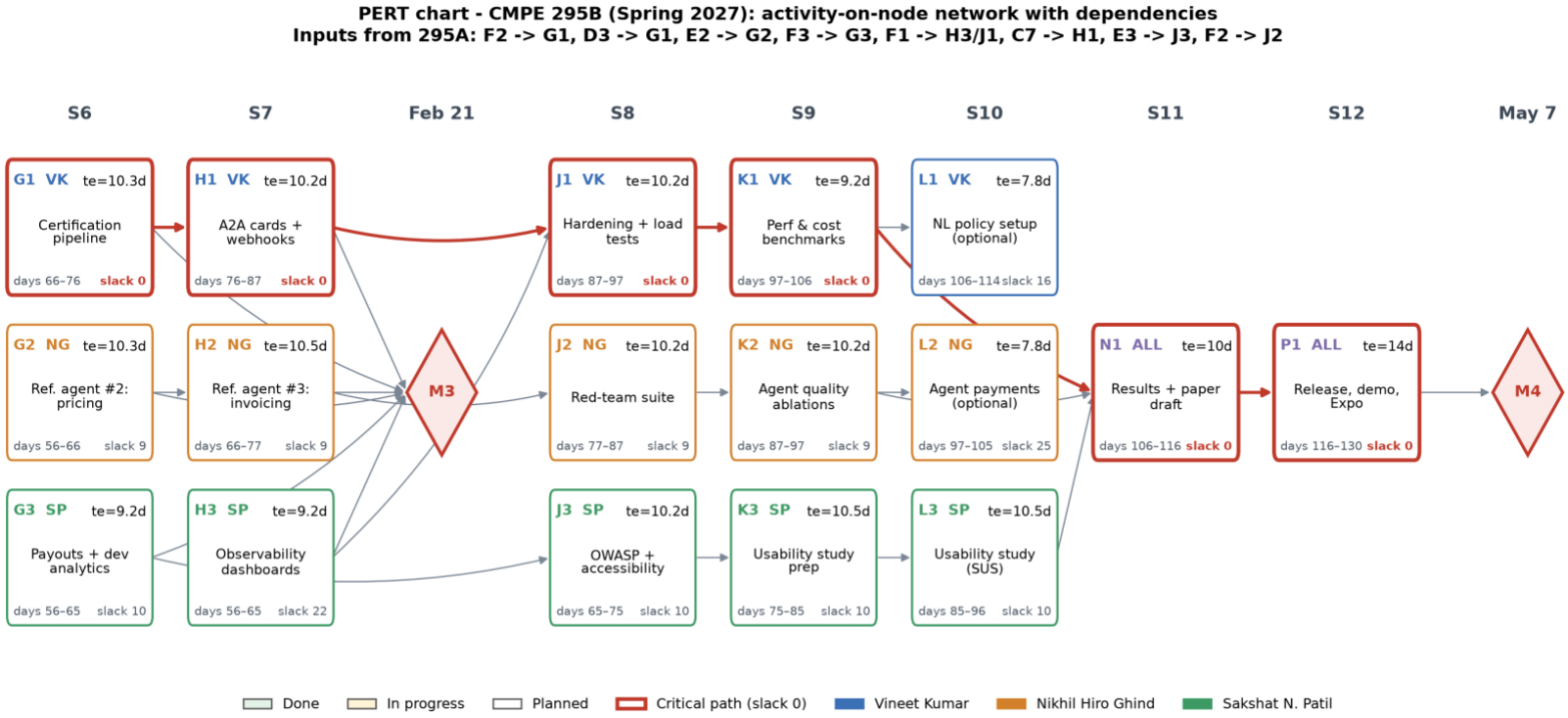


Figure 10. PERT chart for CMPE 295B (Spring 2027) with dependencies and critical path